Warsaw University of Technology

Faculty of Mathematics
and Information Science

Master's diploma thesis

in the field of study

Data Science

Research on Model Compression and Efficiency Optimization Techniques on Deep Learning Models for Edge Deployment

Hubert Jaczyński

student record book number 320597

thesis supervisor

dr hab. Maria Ganzha, prof. PW

WARSAW 2026

Abstract

Research on Model Compression and Efficiency Optimization Techniques on Deep Learning Models for Edge Deployment

Deep learning models achieve high accuracy in computer vision, but their computational and memory demands often prevent direct deployment on resource-constrained edge devices such as cameras, robots, drones, and mobile platforms. This thesis examines practical compression and acceleration techniques for modern object detectors evaluated on the COCO benchmark. The study compares three architectural families: an RF-DETR real-time detection transformer, a Swin-Transformer detector with hierarchical windowed attention, and a YOLO-style convolutional baseline. For each family, the work evaluates precision reduction, INT8 post-training quantisation, token reduction for transformer-style models, and light structured pruning. The proposed evaluation emphasises realistic deployment conditions on an NVIDIA GeForce RTX 3070 Ti and uses consistent metrics, including COCO mAP, AP@0.5, inference latency, model size, FLOPs, and, where feasible, energy-related measurements. The overall goal is to identify which techniques, and which order of applying them, provide the most favourable accuracy–latency–size trade-off for real-time object detection under limited hardware resources.

Keywords: Model compression, object detection, COCO, RF-DETR, Swin Transformer, YOLO, token reduction, quantisation, pruning, edge deployment

Streszczenie

Badania nad technikami kompresji modeli i optymalizacji wydajności modeli głębokiego uczenia się do wdrażania na urządzeniach brzegowych

Modele głębokiego uczenia się osiągają wysoką dokładność w zakresie widzenia komputerowego, ale ich wymagania obliczeniowe i pamięciowe często uniemożliwiają bezpośrednie wdrożenie na urządzeniach brzegowych o ograniczonych zasobach, takich jak kamery, roboty, drony i platformy mobilne. Niniejsza praca bada, w jaki sposób dostosować nowoczesne modele wykrywania obiektów do wnioskowania w czasie rzeczywistym na urządzeniach, zmniejszając ich koszty obliczeniowe przy zachowaniu jakości wykrywania. W pracy dokonano przeglądu głównych rodzin architektur stosowanych w wizji (sieci konwolucyjne i modele oparte na transformatorach), a następnie skupiono się na praktycznych metodach zwiększania wydajności, które można zastosować w przypadku wdrażania na urządzeniach brzegowych. W szczególności zbadano techniki kompresji i przyspieszania modeli, w tym kwantyzację (po szkoleniu i szkolenie uwzględniające kwantyzację), przycinanie (ustrukturyzowane i nieustrukturyzowane) oraz destylację wiedzy, a także podejścia na poziomie architektury, takie jak wyszukiwanie architektury neuronowej uwzględniające sprzęt. Proponowana ocena kładzie nacisk na realistyczne warunki wdrożenia i porównuje metody przy użyciu spójnych wskaźników, w tym dokładności wykrywania, opóźnienia wnioskowania i rozmiaru modelu. Tam, gdzie to możliwe, uwzględnia również aspekty związane z energią. Ogólnym celem jest określenie, które techniki lub kombinacje technik zapewniają najkorzystniejszy kompromis między dokładnością, opóźnieniem i rozmiarem w przypadku wykrywania obiektów w czasie rzeczywistym na ograniczonym sprzęcie oraz w jakim stopniu wyniki zależą od urządzenia docelowego i czasu działania wnioskowania.
Słowa kluczowe: Kompresja modeli, optymalizacja wydajności, głębokie uczenie się, sieci neuronowe, kwantyzacja, przycinanie, destylacja wiedzy, wyszukiwanie architektur neuronowych uwzględniających sprzęt, wdrażanie na urządzeniach brzegowych

Contents

1. Introduction

Deep learning is used in many everyday technologies, especially in systems that analyse images and video [18]. The models behind these systems are improving quickly, but they are also becoming larger and more demanding to run. While powerful servers can handle such models, many real-world applications require the model to run directly on a small device, such as in cameras, robots, drones, or mobile systems [12]. These devices have limited computing power, memory, and battery capacity, so a model that performs well on a workstation may be too slow, too large, or too energy-intensive to be practical on the device [34].

For this reason, there is a growing need to make models lighter and faster while keeping their results as reliable as possible [5]. In practice, this means balancing several factors: the model's accuracy, its prediction speed, the memory it requires, and the energy it consumes. The best balance depends on the intended use case — for example, a safety-critical system may prioritise accuracy, while a battery-powered device may prioritise speed and energy efficiency.

To address these challenges, researchers develop methods that reduce the computational and memory requirements of a model, while aiming to preserve performance as much as possible. Some approaches simplify the model structure, while others reduce the precision of computations, and still others transfer knowledge from a larger model to a smaller one [9]. Additionally, practical performance also depends on the specific hardware and software used to run the model, so it is essential to evaluate solutions under realistic deployment conditions [4].

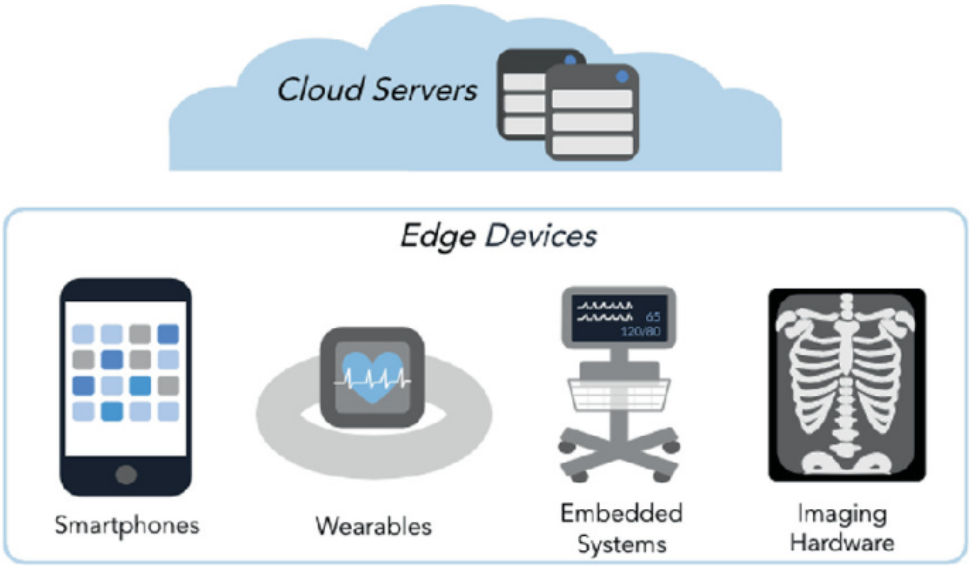


Figure 1.1: Variants of edge devices (reproduced from [1]).

The use of these approaches may have an impact on the application of artificial intelligence in various scenarios. Some of them may not yield any benefit where the highest accuracy is required. Compression may be unacceptable without strict validation in safety-critical settings (eg. medical, automotive).

1.1. Aim of the work

This thesis focuses on real-time object detection under a constrained local hardware budget. The goal is to compare how representative detector families respond to compression and acceleration methods when evaluated on the same dataset, hardware, and measurement protocol. The work concentrates on COCO-trained detectors from three design directions: RF-DETR as a modern real-time detection transformer, Swin Transformer based detection as a hierarchical vision transformer, and YOLO-style detection as a strong convolutional baseline. The YOLO line includes both YOLOv8 and the newer Ultralytics YOLO26 family, which introduces an NMS-free end-to-end inference path aimed at deployment efficiency [31, 22, 15, 16].

The experimental part will not train all detectors from scratch. Instead, it will use published COCO checkpoints and focus on evaluation, export, post-training quantisation, lightweight token reduction, and limited fine-tuning only where it is necessary to recover accuracy. This scope is suitable for a single NVIDIA GeForce RTX 3070 Ti with 8 GB of GPU memory.

The study is guided by the following research questions:

1. How do RF-DETR, Swin Transformer detection, YOLOv8, and YOLO26 compare on the

accuracy–latency–size trade-off when evaluated under the same COCO protocol?

2. Do token-reduction methods preserve detection accuracy better in transformer-style detectors than direct channel or filter reduction does in convolutional detectors?

3. How much accuracy is lost, and how much latency is gained, when moving from FP32 to FP16 and INT8 post-training quantisation for each detector family?

4. Does the order of applying compression methods matter for object detection, for example token reduction followed by INT8 quantisation versus INT8 quantisation followed by token reduction?

5. Which compressed configurations form the best Pareto frontier for deployment on the RTX 3070 Ti, and how different are the conclusions from FLOPs-only estimates?

The expected contribution is an empirical comparison of detector compression strategies under a reproducible local protocol, together with a discussion of which optimisation techniques are most practical for real-time edge-oriented deployment.

2. Related work

The topic addressed in this work belongs to one of the most actively researched areas in modern machine learning. As deep learning systems are deployed more widely, researchers increasingly study how far such models can be made smaller, faster, and less resource-demanding while still producing results that remain useful in practice [5]. This need is driven not only by scientific interest, but also by real engineering constraints: reduced computation can lower deployment costs, enable real-time operation, and make advanced models accessible on devices with limited power, memory, and processing capability [34]. At the same time, the rapid pace of technological change means that the limits of what is feasible continue to shift as new model architectures, optimisation techniques, and hardware platforms emerge [35].

In this thesis, artificial intelligence is considered primarily as a practical tool for automating perception tasks that would otherwise require significant human effort. Specifically, computer vision methods can be employed to detect, classify, or segment objects in images and videos [29, 23]. While these tasks may appear straightforward for humans in many situations, robust performance across different environments, lighting conditions, viewpoints, and object scales remains challenging for machines. Modern deep learning approaches have achieved strong results, but they often require substantial computational resources, which motivates research on methods that reduce inference cost without sacrificing too much accuracy.

The purpose of the related works chapter is therefore to summarise the key directions of research that address efficient inference and deployment. It discusses commonly used approaches to improve efficiency, how these methods affect accuracy and runtime behaviour, and what has been reported in the literature when such techniques are applied to real-time object detection on constrained hardware [28, 9].

2.1. Neural networks

Neural networks are inspired by the way biological nervous systems process information. In their simplest form, they consist of many small computational units (often called *neurons*) that

transform inputs into outputs and can be connected into larger structures. One of the earliest and most influential mathematical models of an artificial neuron was proposed by McCulloch and Pitts in 1943 [24]. In this model, a neuron acts as a threshold-based logic unit: it sums its inputs and produces an output depending on whether that sum exceeds a chosen threshold [24]. This idea later became a foundation for trainable models such as the perceptron and, over time, for more complex architectures [32].

As computing power and training methods improved, neural networks evolved into many specialised families. Examples include recurrent neural networks (RNNs) for sequential data [11], convolutional neural networks (CNNs) for images [19], generative adversarial networks (GANs) for data generation [8], and transformer-based models, which currently dominate many state-of-the-art results in both vision and language tasks [36].

2.1.1. Training neural networks

Training a neural network means finding parameter values (weights and biases) that minimise a chosen objective function [18]. In supervised learning, this objective typically measures the discrepancy between the model predictions and the ground-truth labels. Let a model with parameters θ produce predictions $f(\mathbf{x};\theta)$ for an input $\mathbf{x}$. Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ and a loss function $\ell(\cdot)$, the learning problem is commonly written as

$$\min_{\theta} \; \mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell\big(f(\mathbf{x}_i;\theta), y_i\big),$$

possibly with an additional regularisation term, e.g., $\lambda\|\theta\|_2^2$, to discourage overly large weights and improve generalisation.

Gradient-based optimisation

Modern neural networks are trained using gradient-based methods [18]. Gradients $\nabla_\theta \mathcal{L}(\theta)$ are computed efficiently with backpropagation, which applies the chain rule through the computational graph of the network [33]. A basic optimisation method is gradient descent, which iteratively updates parameters as

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t),$$

where η is the learning rate. In practice, evaluating the full gradient over all N samples at every step is expensive, so training is usually performed with *stochastic gradient descent* (SGD) or its variants [30].

Stochastic gradient descent

In SGD, the gradient is estimated using a mini-batch $\mathcal{B}_t \subset \mathcal{D}$:

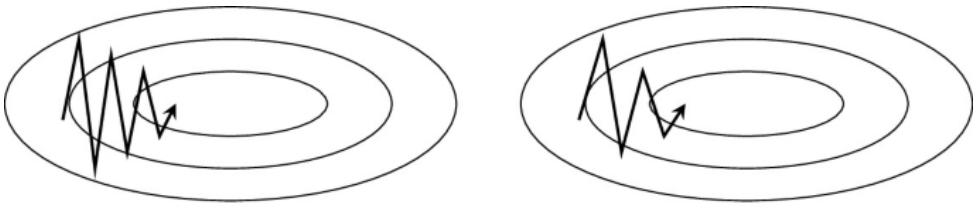
$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}_{\mathcal{B}_t}(\theta_t), \qquad \mathcal{L}_{\mathcal{B}_t}(\theta) = \frac{1}{|\mathcal{B}_t|} \sum_{(\mathbf{x},y)\in\mathcal{B}_t} \ell\big(f(\mathbf{x};\theta), y\big).$$

The stochasticity introduced by mini-batches makes optimisation noisier, but also helps escape poor local regions and enables training on large datasets. Common improvements to SGD include momentum and weight decay. Momentum accumulates a running direction of descent [26]:

$$\mathbf{v}_{t+1} = \mu \mathbf{v}_t + \nabla_\theta \mathcal{L}_{\mathcal{B}_t}(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta \mathbf{v}_{t+1},$$

where $\mu \in [0, 1)$ controls how strongly past gradients influence the update.

The convergence to the minimum is shown in Figure 2.1



(a) Gradient descent (b) Gradient descent with momentum

Figure 2.1: Stochastic Gradient Descent comparison (reproduced from [25]).

Learning rate selection and line search intuition

The learning rate is one of the most important hyperparameters: if it is too large, the optimisation may diverge; if it is too small, convergence can be slow [18]. In classical optimisation, *line search* methods choose a step size that sufficiently decreases the objective (for example, through the Armijo condition). Although full line search is rarely used in large-scale deep learning due to its computational cost, the same intuition is evident in practice through learning rate schedules, warm-up phases, and adaptive strategies.

A related idea is *backtracking*: when an update increases the loss or causes unstable training, the step size is reduced and the update is retried. While this is not always implemented explicitly, it is conceptually similar to common training practices such as reducing the learning rate on a plateau, using gradient clipping to avoid exploding updates, or using early stopping to prevent overfitting [18].

Searching for good solutions

Neural network training is a non-convex optimisation problem, meaning there may be many parameter configurations that achieve low loss but differ in generalisation performance and robustness. As a result, training often involves a search process over:

- **Optimiser choice** (e.g., SGD with momentum, Adam, AdamW),
- **Learning rate schedules** (step decay, cosine decay, one-cycle policy),
- **Regularisation** (weight decay, data augmentation, dropout),
- **Model capacity** (number of layers/channels, feature dimensions).

These choices influence not only final accuracy but also how well a model transfers to constrained environments, such as edge devices [34].

2.2. Deep neural networks

A deep neural network is a neural network composed of multiple successive layers, where each layer applies a learned transformation followed by a non-linear activation [18]. In contrast to shallow models, depth allows the network to represent complex functions through the composition of simpler operations. In computer vision, this often leads to hierarchical feature representations: early layers learn low-level patterns, such as edges and textures, while deeper layers capture higher-level structures, including object parts and entire objects [18].

Despite their strong performance, deep models can be computationally demanding [34]. The cost of inference is determined not only by the number of parameters (which affects storage) but also by the number of operations and the memory required to store intermediate activations. In edge deployments, limitations in processing power, memory capacity, and energy availability make these factors critical. In particular, memory access and bandwidth can become a bottleneck, meaning that reductions in model size or operation count do not always translate directly into proportional speed-ups on a given device [34].

These properties motivate the use of efficiency techniques that aim to reduce resource requirements while preserving accuracy [5]. Methods such as reduced numerical precision, removal of redundant parameters or structures, transferring knowledge from larger models, and automatically searching for efficient architectures are therefore central to deploying deep networks under real-world constraints [7, 9].

2.3. Convolutional neural networks

Convolutional Neural Networks (CNNs) are a class of deep learning models designed primarily for processing data with a grid-like structure, most notably images [18]. Their key idea is to exploit the spatial structure of an image by using *local connectivity* and *weight sharing*. Unlike fully connected layers, which learn a separate weight for every input feature, convolutional layers apply the same small set of weights (a kernel) across the entire image. This reduces the number of parameters and makes the model more efficient, while also providing a degree of *translation equivariance*, meaning that shifting an object in the image results in a corresponding shift in the feature map. An example architecture is shown in Figure 2.2.

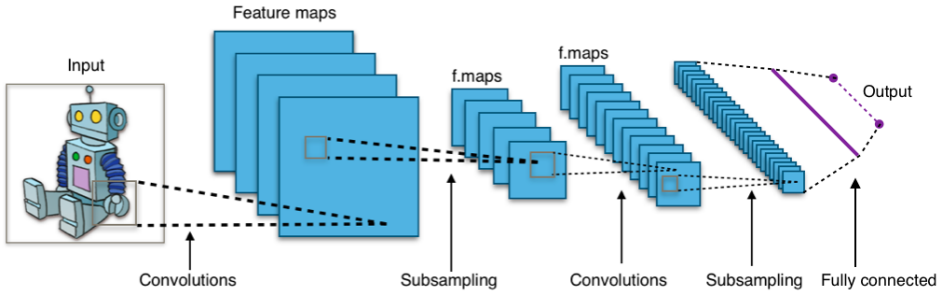


Figure 2.2: CNN architecture visualisation. Reproduced from Aphex34 [2] (CC BY-SA 4.0).

2.3.1. Convolution operation

Given an input feature map $X \in \mathbb{R}^{H \times W \times C_{\text{in}}}$, a convolutional layer produces an output feature map $Y \in \mathbb{R}^{H' \times W' \times C_{\text{out}}}$ by sliding learnable filters over the spatial dimensions. For a filter of size $k \times k$, the output at spatial location (u, v) and output channel j can be written as

$$Y_{u,v,j} = b_j + \sum_{c=1}^{C_{\text{in}}} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} W_{p,q,c,j} \, X_{u+p,\, v+q,\, c}, \tag{2.1}$$

where W denotes the convolutional kernel weights and b_j is the bias for channel j. In practice, convolutions often use a stride s (skipping positions) and padding P (extending borders) to control the spatial resolution of the output [18].

Convolution is usually followed by a non-linear activation such as ReLU,

$$\text{ReLU}(x) = \max(0, x),$$

which enables the network to model non-linear decision functions. Stacking multiple convolutional layers allows the network to build increasingly abstract features.

2.3.2. Pooling and hierarchical representations

Many CNN architectures include pooling operations, such as max pooling, to reduce spatial resolution and increase robustness to small shifts and distortions [18]. For example, max pooling with window size $k \times k$ can be expressed as

$$Y_{u,v,c} = \max_{(p,q)\in\Omega} X_{u+p,\, v+q,\, c},$$

where Ω indexes the pooling window. Modern architectures may replace pooling with strided convolutions, but the purpose is similar: gradually reducing resolution while increasing the number of channels, forming a hierarchical representation [18].

2.3.3. Typical CNN building blocks

Over time, CNNs have evolved from simple stacks of convolution and pooling layers into more advanced building blocks [18]:

- **Residual connections** (e.g., ResNet), which add skip paths to improve gradient flow and enable deeper networks.
- **Normalisation layers** (e.g., Batch Normalisation), which stabilise training and often speed up convergence.
- **Bottleneck designs**, which use 1×1 convolutions to reduce and then restore channel dimensionality, decreasing computation.

These design choices are important for practical efficiency, especially when models are deployed on limited hardware [34].

2.3.4. Efficiency considerations for edge deployment

CNNs are widely used for edge inference because their operations are structured and can be implemented efficiently on GPUs, NPUs, and other accelerators [34]. However, CNNs can still be computationally expensive, particularly in early layers that process high-resolution feature maps. The total inference cost is influenced by:

- the input resolution (H, W) and the number of channels,
- kernel size and stride,
- the number of layers and output channels,
- memory bandwidth required to read and write feature maps.

A common proxy for compute is the number of multiply–accumulate operations (MACs) [34]. For a standard 2D convolution, the approximate MAC count is

$$\text{MACs} \approx H'W' \cdot C_{\text{out}} \cdot (k^2 C_{\text{in}}).$$

This expression highlights why reducing resolution, kernel size, or channel counts can strongly affect runtime [34].

2.3.5. CNNs and model compression

CNN structure makes them suitable for many compression methods discussed later in this thesis [5]. For example, *structured pruning* can remove entire channels or filters, directly reducing computation [9]. *Quantisation* can reduce the precision of weights and activations (e.g., from FP32 to INT8), lowering memory usage and often improving latency on supported hardware [7]. Furthermore, efficient CNN variants (e.g., based on depthwise separable convolutions and other factorised operations) are often designed specifically to reduce MACs and memory access, which is particularly beneficial for real-time object detection on edge devices [12].

Overall, CNNs remain a foundational architecture in computer vision. Even when newer paradigms, such as transformers, are used, CNN-like components are often retained in early stages or in hybrid models due to their efficiency and strong inductive bias for local spatial patterns [18].

2.4. Transformers

Transformers are a class of neural network architectures originally introduced for sequence modelling tasks [36]. Their main idea is to replace recurrence with an attention mechanism that enables the model to directly relate different parts of the input to one another. This makes transformers highly parallelisable during training and enables them to capture long-range dependencies more effectively than many earlier architectures [36].

2.4.1. Self-attention mechanism

The core component of a transformer is *self-attention* [36]. Given an input sequence represented by embeddings

$$X \in \mathbb{R}^{n \times d},$$

where n is the number of tokens and d is the embedding dimension, the model computes three learned projections: queries Q, keys K, and values V:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ are trainable matrices. Attention weights are then computed using scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \tag{2.2}$$

Intuitively, the softmax term determines how strongly each token should attend to all other tokens. This allows the representation of each token to be updated based on information from the entire sequence [36].

In practice, transformers use *multi-head attention*, where multiple attention operations are performed in parallel with different learned projections [36]. This enhances expressiveness by enabling the model to simultaneously focus on different types of relationships. The outputs of the heads are concatenated and projected back to the model dimension:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O.$$

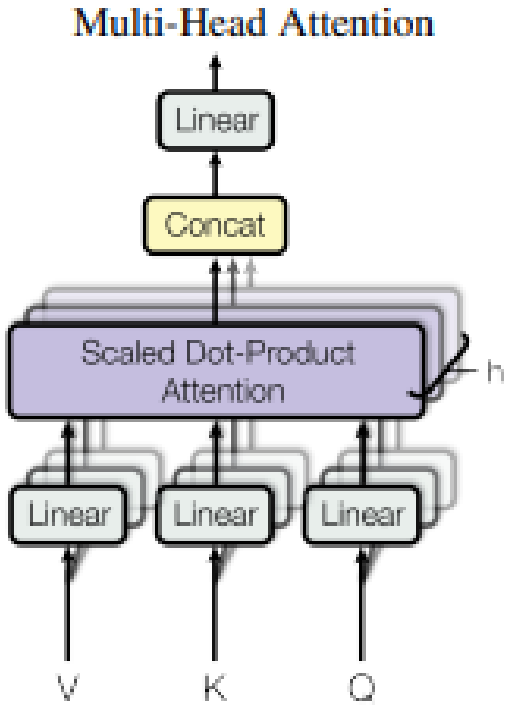


Figure 2.3: Multi-head attention overview (reproduced from [36]).

2.4.2. Transformer block

A standard transformer encoder block consists of:

- a multi-head self-attention layer,
- a position-wise feed-forward network (FFN),
- residual connections and normalisation layers.

A simplified formulation of an encoder block can be written as

$$X' = \mathrm{LN}\big(X + \mathrm{MHA}(X)\big), \qquad Y = \mathrm{LN}\big(X' + \mathrm{FFN}(X')\big),$$

where $\mathrm{LN}(\cdot)$ denotes layer normalization. The feed-forward network is typically implemented as two linear layers with a non-linear activation:

$$\mathrm{FFN}(x) = W_2\sigma(W_1 x + b_1) + b_2.$$

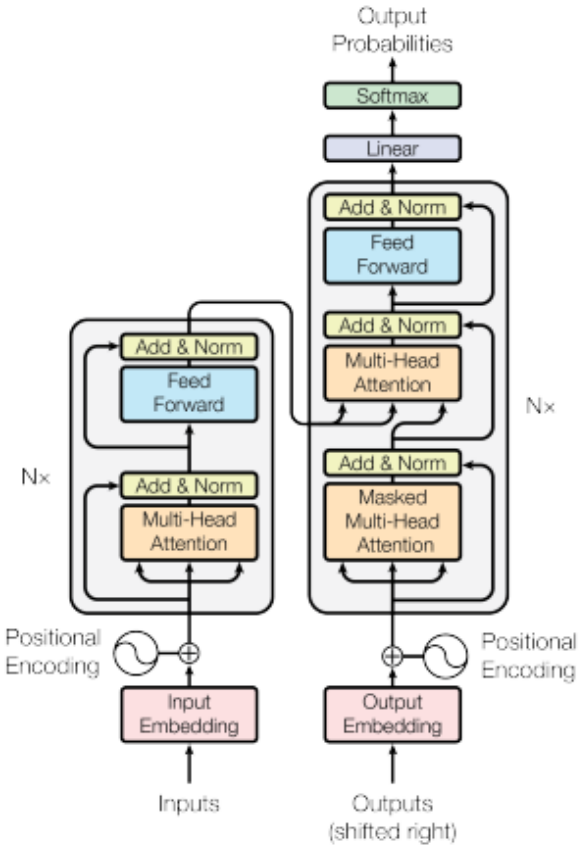


Figure 2.4: Transformer architecture overview (reproduced from [36]).

2.4.3. Efficiency considerations

Compared to convolutions, the attention mechanism has a different computational profile [36]. For a sequence of length n, the QK^T operation in Equation (2.2) has $\mathcal{O}(n^2)$ complexity, which can become expensive when n is large [36]. In many practical settings, runtime and memory are influenced not only by arithmetic operations but also by the cost of moving data between memory and compute units [34]. These factors are particularly important for edge deployment, where memory bandwidth and cache sizes are limited [34].

2.4.4. Relation to computer vision

Transformers were originally designed for language processing, where inputs are naturally represented as token sequences [36]. Images, however, are two-dimensional arrays of pixels. To apply transformers to vision tasks, images must be converted into a sequence representation while preserving spatial information. This motivates Vision Transformers, introduced in the next subsection [6].

2.4.5. Vision Transformers

Vision Transformers (ViTs) adapt transformer encoders to image understanding by representing an image as a sequence of fixed-size patches [6]. An image of size $H \times W$ is split into patches of size $P \times P$, producing

$$n = \frac{HW}{P^2}$$

patches. Each patch is flattened and mapped into a d-dimensional embedding space. Because transformers have no inherent notion of token order, learnable *positional embeddings* are added to encode patch locations [6]. The resulting sequence is then processed by a standard transformer encoder stack [6].

ViTs have demonstrated strong performance, particularly when trained on large datasets, and are utilised in many modern vision systems [6]. However, the quadratic cost of attention with respect to the number of patches means that high-resolution inputs can be computationally expensive [36]. As a result, many practical vision transformer variants introduce efficiency improvements, such as hierarchical token representations, local or windowed attention, and reduced token counts [22]. These considerations are directly relevant to edge deployment, where input resolution and model size strongly affect latency and energy consumption [34].

2.5. Object detection architectures considered in this work

Object detection extends image classification by requiring both class prediction and spatial localisation. Modern detectors therefore differ not only in the feature extractor used to represent an image, but also in the way they assign predictions to objects, decode bounding boxes, and perform post-processing. Since these design choices influence both accuracy and runtime, this thesis compares detector families rather than isolated backbones.

2.5.1. YOLO-style one-stage detectors

YOLO introduced a one-stage formulation in which object classes and bounding boxes are predicted directly from an image in a single forward pass [28]. This idea is attractive for real-time deployment because it avoids the separate region-proposal stage used by two-stage detectors such as Faster R-CNN [29]. Later YOLO implementations improved the training recipe, feature aggregation, anchor handling, loss functions, export tooling, and hardware deployment support. In this thesis, YOLOv8 is used as a mature Ultralytics baseline, while YOLO26 is included as a newer comparison point intended to represent the current direction of deployment-oriented YOLO models [15, 16].

2.5.2. Detection transformers and RF-DETR

DETR-style models formulate object detection as a set prediction problem, replacing many hand-designed post-processing components with transformer attention and learned object queries. This family is conceptually important because it connects object detection with the broader success of transformer architectures in vision and language [36, 6]. RF-DETR is especially relevant for this thesis because it targets the real-time part of the design space: instead of only maximising accuracy, it searches for accuracy–latency Pareto-efficient detection transformer variants and provides COCO-pretrained checkpoints suitable for practical evaluation [31]. It therefore serves as the main transformer detector baseline in the experimental part.

2.5.3. Swin Transformer detectors

Swin Transformer addresses a key limitation of plain ViTs for dense vision tasks: global attention over all image patches becomes expensive at high resolution. Swin uses a hierarchical representation and shifted local windows, which reduces attention cost while still allowing information to move across neighbouring windows [22]. This makes it suitable for object detection and segmentation, where multi-scale spatial features are important. In this thesis, a Swin-T

Mask R-CNN detector is treated as the hierarchical vision-transformer representative. It is not expected to be the fastest model, but it provides a useful contrast to both RF-DETR and YOLO: it combines transformer-style feature extraction with a conventional detection framework.

2.6. Token reduction for vision transformers

Token reduction is an efficiency direction specific to transformer-style models. In a vision transformer, an image is represented as a sequence of patch tokens. Since self-attention compares tokens with one another, reducing the number of tokens can lower both computation and memory use, especially in deeper layers where the representation is already semantically richer [36, 6]. This makes token reduction complementary to quantisation and pruning: quantisation reduces numerical precision, pruning removes model structures or parameters, while token reduction reduces the amount of intermediate representation processed during inference.

2.6.1. Token pruning and token merging

Token-reduction methods can be broadly divided into token pruning and token merging. Token pruning removes tokens judged to be less important, for example background patches or low-saliency regions. This can be efficient, but it risks discarding information that later layers might need. Token merging instead combines similar or redundant tokens into fewer representative tokens. This may preserve more information than hard removal because the merged token still carries a fused representation of the original tokens.

Token Merging (ToMe) is a representative training-free approach that merges similar tokens inside ViT models to reduce inference cost without retraining the full network [3]. Its appeal for this thesis is practical: it suggests that token reduction can be evaluated as an inference-time modification before committing to expensive fine-tuning. More recent methods such as MaMe and MPM further explore how token pairs or groups should be selected and fused [13, 27]. Although these works are not yet standard deployment tools for object detection, they provide useful design ideas for reducing token count in RF-DETR and Swin-style detectors.

2.6.2. Why detection is harder than classification

Applying token reduction to object detection is more difficult than applying it to image classification. A classifier only needs enough information to predict one or more image-level labels. A detector must also localise objects, preserve small-object evidence, and maintain spatial detail across the feature hierarchy. Removing or merging tokens too aggressively may therefore hurt

bounding-box quality even if the main object category remains recognisable.

This is particularly important for COCO, where many images contain multiple objects at different scales. Background-looking tokens may still provide context, and small objects may occupy only a few patches. For this reason, token reduction for detection should start conservatively: lower merge ratios, later-layer merging, and careful measurement of AP for small, medium, and large objects. The final evaluation should not only report global mAP, but also check whether small-object AP degrades disproportionately.

2.6.3. Relation to RF-DETR and Swin

RF-DETR and Swin provide two different settings for token reduction. In RF-DETR, token reduction can be inserted in the encoder before object queries attend to image features. The challenge is to reduce redundant image tokens without weakening the decoder's ability to identify object locations. In Swin, token reduction must respect the hierarchical windowed structure. A merge applied inside local windows is easier to align with Swin's attention pattern, while merging across windows may reduce more redundancy but risks disturbing the shifted-window design [22].

For this thesis, token reduction is therefore treated as a planned transformer-specific compression axis rather than as a completed result. The initial implementation should use a simple ToMe-like or MaMe/MPM-inspired merging step, measure its isolated effect, and only then combine it with FP16 or INT8. This order is important because token reduction changes the representation itself, while quantisation changes numerical precision. Evaluating them separately makes the source of any accuracy loss easier to identify.

2.7. Frugal artificial intelligence

As deep learning systems become more widely deployed, practical limitations such as computation time, memory usage, energy consumption, and data availability increasingly determine whether a solution can be used in real-world scenarios. In many applications, especially on edge devices, it is not sufficient for a model to be accurate in a laboratory setting—it must also be efficient enough to run under strict hardware constraints and, in some cases, with limited access to training data or communication bandwidth. These constraints motivate a set of methods whose goal is to reduce the overall "cost" of using artificial intelligence while keeping performance as close as possible to that of unconstrained models [5].

In this thesis, the term *frugal artificial intelligence* refers to the design and deployment of AI systems that aim to achieve strong predictive performance while minimising resource require-

ments [34]. This includes reducing the computational cost of inference, lowering memory and storage demands, and improving energy efficiency [34]. In addition, frugal approaches can also address data-related costs, such as the amount of labelled data needed for training or the cost of transmitting data to remote servers. The underlying idea is to build systems that are not only accurate but also feasible to deploy and maintain in environments with limited resources.

Frugal AI is particularly relevant for real-time computer vision tasks such as object detection, where models must process high-dimensional inputs (images or video frames) within strict time budgets [28]. In such settings, the core challenge is to navigate trade-offs between accuracy and efficiency. The following subsections introduce the main families of techniques used to make deep learning models more suitable for constrained deployment [5]. These include reducing numerical precision, removing redundant parameters or structures, transferring knowledge from larger models to smaller ones, and automatically searching for architectures that satisfy hardware-dependent requirements [7, 10, 4].

2.7.1. Quantisation

Quantisation is a family of techniques that reduce the numerical precision used to represent a neural network. In standard deep learning workflows, model weights and activations are typically stored and computed in 32-bit floating point (FP32). Although FP32 offers high numerical accuracy, it requires significant memory bandwidth and compute [34]. Quantisation replaces FP32 with lower-precision formats such as FP16 or INT8, which can reduce model size and, on supported hardware, improve inference speed and energy efficiency. This makes quantisation particularly relevant for edge deployment, where memory and power are often limited.

Basic idea

In many practical schemes, a real-valued tensor x is mapped to an integer tensor x_q through an affine transformation defined by a *scale* s and a *zero-point* z [14]:

$$x_q = \text{clip}\left(\left\lfloor \frac{x}{s} \right\rceil + z,\ q_{\min},\ q_{\max} \right), \tag{2.3}$$

where $\lfloor \cdot \rceil$ denotes rounding to the nearest integer and $[q_{\min}, q_{\max}]$ defines the representable integer range (for example $[-128, 127]$ for signed INT8). During inference, values can be approximately reconstructed by

$$\hat{x} = s(x_q - z). \tag{2.4}$$

The parameters s and z are commonly derived from observed tensor ranges. If x is assumed to lie in $[x_{\min}, x_{\max}]$, a typical choice is

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \qquad z = \left\lfloor q_{\min} - \frac{x_{\min}}{s} \right\rceil .$$

This formulation is widely used in deployment toolchains because it enables efficient integer arithmetic while approximating the original floating-point computations [14].

Weight quantisation and activation quantisation

Quantisation can be applied to:

- **Weights** (model parameters), which primarily reduces storage and improves cache efficiency. Weight-only quantisation can already provide benefits in memory-constrained scenarios.
- **Activations** (intermediate feature maps), which often dominate runtime memory usage during inference, especially for high-resolution vision models. Quantising activations can substantially reduce memory bandwidth requirements.

In practice, quantising activations is more challenging than quantising weights, because activation distributions depend on the input data and can vary between layers [7].

Granularity: per-tensor vs. per-channel

A key design choice is the granularity at which quantisation parameters are defined [7]:

- **Per-tensor quantisation**: a single (s, z) pair is used for the entire tensor. This approach is simple and efficient, but it can compromise accuracy if the ranges of different channels differ significantly.
- **Per-channel quantisation**: separate scales (and sometimes zero-points) are used for each output channel (common for convolution weights). This typically improves accuracy because it adapts to channel-wise variation at a small additional implementation cost.

For convolutional layers, per-channel quantisation of weights is commonly used in practice because it often yields better accuracy at the same bit-width.

Post-training quantisation (PTQ)

Post-training quantisation converts a trained FP32 model to lower precision without retraining the model. The main advantage of PTQ is that it is fast and easy to apply. A typical

PTQ workflow includes a *calibration* step, where a small representative dataset is passed through the network to estimate activation ranges. PTQ is widely used in deployment scenarios when re-training is not feasible [7].

However, PTQ may introduce a noticeable decrease in accuracy for certain models and tasks. Object detection is often more sensitive than image classification because it includes both classification and localisation components and may depend on small numerical differences in bounding box regression.

Quantisation-aware training (QAT)

Quantisation-aware training incorporates quantisation effects during training, allowing the network to adapt to reduced precision. A common approach uses *fake quantisation*: during the forward pass, tensors are quantised and dequantised to simulate INT8 behaviour, while gradients are still computed in floating-point (often using a straight-through estimator). As a result, QAT typically achieves higher accuracy than PTQ at the same bit-width, at the cost of additional training time and complexity [7].

Symmetric vs. asymmetric quantisation

Quantisation can be [7]:

- **Symmetric**, where $z = 0$ and the integer range is centred around zero. This can simplify computation and is common for weights.
- **Asymmetric**, where $z \neq 0$, allowing the range to shift. This can better represent non-zero-centred distributions and is often used for activations.

The choice affects both accuracy and hardware efficiency, and in practice, it is often determined by the deployment framework and accelerator capabilities [7].

Why quantisation helps on edge devices

Quantisation can improve edge inference for several reasons [34]:

- **Reduced model size**: fewer bits per weight lowers storage requirements and improves cache usage.
- **Lower memory bandwidth**: quantised activations require fewer bytes to read and write, which can be critical on constrained devices.

- **Faster compute on supported hardware**: many NPUs and mobile accelerators provide highly optimised INT8 operations.
- **Lower energy consumption**: moving and processing fewer bits often reduces power usage, which is important for battery-powered systems.

At the same time, observed speedups depend on the hardware and runtime stack. If the inference engine does not support a quantised operator, it may fall back to floating-point execution, reducing the practical benefits [34].

Quantisation in the context of compression pipelines

Quantisation is often combined with other compression methods [5]. For example, pruning can reduce the number of channels or filters, and quantisation can then further reduce memory and bandwidth [9]. Knowledge distillation can help recover accuracy lost due to low-precision constraints by training a compact quantised model to mimic a larger teacher [10]. These combinations are commonly used in edge deployment workflows because they address different sources of cost: model size, compute, and numerical precision [5].

2.7.2. Pruning

Pruning refers to a family of methods that reduce the size and computational cost of a neural network by removing parameters that are considered unnecessary [5]. Modern deep learning models are often over-parameterised, meaning that they contain redundancy in their weights and internal representations. Pruning attempts to exploit this redundancy by eliminating parts of the model while maintaining accuracy as much as possible. In the context of edge deployment, pruning is attractive because it can reduce inference latency, memory usage, and energy consumption, especially when it leads to a smaller dense model that runs efficiently on standard hardware [34].

General formulation

Let a model be parameterised by $\theta \in \mathbb{R}^n$. A common view of pruning is to introduce a binary mask $\mathbf{m} \in \{0,1\}^n$ that selects which parameters remain active:

$$\theta' = \mathbf{m} \odot \theta,$$

where $\odot$ denotes element-wise multiplication and θ' are the effective parameters used at inference. Pruning corresponds to setting some components of $\mathbf{m}$ to zero. The resulting pruned model is then typically fine-tuned to recover performance [9].

Unstructured pruning

Unstructured pruning removes individual weights, for example, by setting to zero weights with small magnitude [9]:

$$m_i = \begin{cases} 0, & \text{if } |\theta_i| < \tau, \\ 1, & \text{otherwise,} \end{cases}$$

where τ is a threshold chosen to reach a desired sparsity level. This approach can substantially reduce the number of non-zero parameters, but the remaining weights form irregular sparse patterns. In practice, unstructured sparsity does not always translate into lower latency on edge devices because efficient execution often requires specialised sparse kernels or hardware support. Without such support, a sparse model may be stored compactly but still executed as if it were dense [34].

Structured pruning

Structured pruning removes whole structures such as convolutional channels, filters, attention heads, or complete blocks. This produces a smaller dense network with reduced tensor dimensions, which typically provides more predictable speedups on common accelerators [34].

For example, in a convolutional layer, one may prune output channels (filters). If the original convolution has C_{out} output channels, pruning removes a subset of these channels and adjusts subsequent layers accordingly. This directly reduces both parameter count and computational cost, since the approximate MAC count of a convolution scales with channel dimensions:

$$\text{MACs} \approx H'W' \cdot C_{\text{out}} \cdot (k^2 C_{\text{in}}).$$

Reducing C_{out} (and often C_{in} in downstream layers) therefore reduces compute and memory traffic [34].

Pruning criteria

Pruning methods differ mainly in how they decide what to remove. Common criteria include [5]:

- **Magnitude-based pruning**: removes weights or channels with small magnitude (simple and widely used).
- **Gradient-based or sensitivity-based pruning**: removes parameters that have a small effect on the loss, often estimated using gradients or second-order approximations.

- **Norm-based channel pruning**: removes channels with small ℓ_1 or ℓ_2 norm in convolution kernels.

In practice, magnitude- or norm-based pruning is popular because it is easy to implement and often provides a good baseline.

One-shot vs. iterative pruning

Pruning can be applied:

- **One-shot**, where a fixed fraction of parameters is removed at once, followed by fine-tuning.
- **Iteratively**, where pruning is performed gradually over multiple stages (e.g., prune a small fraction, fine-tune, repeat). This often yields better accuracy for the same final sparsity because the network has time to adapt [9].

Iterative pruning is especially common when aggressive compression is required, such as for real-time inference on constrained devices.

Pruning and fine-tuning

After pruning, fine-tuning is usually required [9]. Pruning changes the model capacity and can disrupt learned representations. Fine-tuning allows the remaining parameters to adapt and compensate for removed components. In some workflows, pruning is integrated into training by adding sparsity-inducing regularisation, for example, an ℓ_1 penalty on weights or structured regularisers that encourage entire channels to become small.

Pruning for edge deployment

For edge deployment, the practical benefit of pruning depends strongly on whether pruning reduces the *actual* runtime bottlenecks [34]. Structured pruning typically provides more consistent improvements because it reduces tensor sizes and can be accelerated by standard dense kernels. In contrast, unstructured pruning may yield a smaller parameter count without latency gains unless the target device and inference engine support sparse computation [34].

When applied to object detection models, pruning must be performed carefully because detection performance can be sensitive to capacity reductions, especially for small objects and complex scenes [5]. Common practice is to primarily prune the backbone and neck components (which dominate computation) while preserving critical parts of the detection head. In many cases, pruning is combined with other methods such as quantisation or knowledge distillation to recover accuracy while maintaining a smaller and faster model.

2.7.3. Knowledge distillation

Knowledge distillation is a training strategy used to improve the performance of a compact model by transferring information from a larger, typically more accurate model. The larger model is referred to as the *teacher*, while the smaller model is the *student*. The primary motivation is that small models often have limited capacity and may struggle to achieve high accuracy when trained solely with hard ground-truth labels. A teacher model can provide richer supervision signals that help the student learn a better decision function without increasing inference cost [10].

Teacher–student framework

In the teacher–student setup, the teacher network $f_T(\cdot)$ is trained first (or chosen as an already strong model). The student network $f_S(\cdot)$ is then trained using both:

- **hard labels** (the original targets), and
- **soft targets** derived from the teacher outputs.

Soft targets contain information about class similarities and uncertainties. For example, if the teacher assigns non-zero probability to multiple classes, it communicates that these classes are visually related, which can improve generalisation compared to training only on one-hot labels [10].

Logit distillation and temperature scaling

A common form of distillation uses the teacher and student logits (pre-softmax outputs). Let $\mathbf{z}_T$ and $\mathbf{z}_S$ denote teacher and student logits for the same input. Their softened probability distributions are computed with a temperature parameter $\tau > 1$:

$$p_T(\tau) = \mathrm{softmax}\left(\frac{\mathbf{z}_T}{\tau}\right), \qquad p_S(\tau) = \mathrm{softmax}\left(\frac{\mathbf{z}_S}{\tau}\right). \tag{2.5}$$

Higher τ produces a softer distribution, revealing more information about the teacher's relative preferences among classes. The student is trained by minimising a weighted combination of the standard supervised loss and the distillation loss:

$$\mathcal{L} = (1-\lambda)\,\mathcal{L}_{\mathrm{sup}}(y, p_S(1)) + \lambda\,\tau^2\,\mathrm{KL}\big(p_T(\tau)\,\|\,p_S(\tau)\big), \tag{2.6}$$

where $\lambda \in [0,1]$ balances the two objectives, and $\mathrm{KL}(\cdot\|\cdot)$ is the Kullback–Leibler divergence. The τ^2 term is commonly used to maintain comparable gradient magnitudes across different temperature choices [10].

Distillation beyond logits

While logit distillation is widely used for classification, distillation can also transfer information from intermediate representations. Common extensions include:

- **Feature distillation**: the student is encouraged to match the teacher's feature maps at selected layers.
- **Attention distillation**: the student matches attention maps or activation patterns, which can guide where the model should focus.
- **Relation distillation**: the student learns relationships between samples or regions that are captured by the teacher.

These variants can be particularly helpful when the student architecture differs significantly from the teacher [10].

Knowledge distillation for object detection

Object detection is more complex than classification because the model must solve multiple sub-tasks, typically including classification and localisation of bounding boxes. As a result, distillation for detection often combines multiple signals, such as:

- distillation of classification scores,
- distillation of bounding box regression outputs,
- distillation of intermediate features (e.g., backbone or neck feature maps),
- distillation of region-level predictions (for two-stage detectors) or dense prediction maps (for one-stage detectors).

Because the output structure is richer, detection distillation methods must be designed carefully to avoid transferring teacher errors or overly constraining the student [29].

Why distillation is useful for compression

Distillation is especially useful in compression pipelines because it can reduce the accuracy gap between a compact model and a large baseline without increasing inference cost [5]. This makes it complementary to other efficiency methods:

- A **pruned** model can be fine-tuned with distillation to recover performance lost by removing channels or filters.

- A **quantised** model can be trained with distillation so that reduced precision causes smaller accuracy degradation.
- A **lightweight student architecture** (manually designed or found by NAS) can be distilled from a stronger teacher to improve accuracy under strict latency or energy constraints.

In edge deployment scenarios, where model capacity is limited, distillation is often one of the most effective ways to maintain accuracy while meeting runtime budgets [34].

Limitations and practical considerations

Although distillation can significantly improve compact models, it also introduces additional training complexity [10]. A strong teacher is required, and the choice of distillation targets, temperature τ, and weighting λ can influence results. Moreover, the student may inherit some of the teacher's biases or failure modes. For these reasons, distillation is typically evaluated in conjunction with other compression techniques under consistent deployment conditions [5].

2.7.4. Neural Architecture Search and Hardware-aware design

Neural Architecture Search (NAS) is a family of methods that automate the design of neural network architectures [4]. Instead of manually choosing the number of layers, channel widths, kernel sizes, or attention configurations, NAS explores a predefined *search space* of candidate architectures and aims to identify models that achieve strong performance. This approach is particularly relevant in the context of efficient deployment, because the architecture itself strongly influences inference latency, memory usage, and energy consumption [34].

Problem formulation

NAS can be expressed as an optimisation problem over architectures α from a search space $\mathcal{A}$. The goal is often to maximise predictive performance while satisfying efficiency constraints:

$$\max_{\alpha \in \mathcal{A}} \ \mathrm{Acc}(\alpha) \quad \text{subject to} \quad \mathrm{Cost}(\alpha) \leq C_{\max}, \tag{2.7}$$

where $\mathrm{Cost}(\alpha)$ can represent latency, parameter count, memory usage, energy consumption, or a combination of these metrics. In many edge scenarios, the constraint is expressed directly as a latency budget measured on a target device [4].

Alternatively, the objective may be written as a multi-objective trade-off, for example maximising accuracy while penalising cost:

$$\max_{\alpha \in \mathcal{A}} \ \mathrm{Acc}(\alpha) - \beta \, \mathrm{Cost}(\alpha),$$

where β controls how strongly efficiency is prioritised.

Search strategies

NAS approaches differ mainly in the strategy used to explore the search space. Common strategies include:

- **Evolutionary search**, where architectures are mutated and selected over generations.
- **Reinforcement learning**, where a controller proposes architectures and receives rewards based on performance.
- **Gradient-based NAS**, where a continuous relaxation of the search space allows optimisation by gradient descent.

In addition to the search strategy, NAS also depends on how candidate architectures are evaluated, which is often the dominant computational cost [4].

Hardware-aware design

In practice, theoretical measures such as FLOPs do not always predict real latency on a specific device. Hardware-aware methods, therefore, incorporate device-dependent measurements or predictors. A hardware-aware formulation replaces $\mathrm{Cost}(\alpha)$ with a metric measured on a target platform (or estimated by a surrogate model):

$$\mathrm{Cost}(\alpha) = \mathrm{Lat}_{\mathrm{device}}(\alpha) \quad \text{or} \quad \widehat{\mathrm{Lat}}_{\mathrm{device}}(\alpha).$$

This is important because practical runtime is influenced by factors such as memory bandwidth, kernel implementations, operator fusion, and the inference engine [34]. For edge deployment, hardware-aware NAS can therefore produce architectures that are efficient *in practice*, not only on paper.

Computational requirements and practical limitations

A major drawback of NAS is its computational cost. Evaluating many candidate architectures can require significant resources, often involving repeated training or fine-tuning. Early NAS approaches were prohibitively expensive, sometimes requiring hundreds or thousands of GPU-days. Although modern methods have improved efficiency (e.g., weight sharing, early stopping, or proxy tasks), NAS can still be difficult to perform under limited computational budgets [4].

For this reason, when compute resources are constrained, NAS is often used in a reduced or practical form, for example:

- restricting the search space to a small set of design choices,

- using pre-trained backbones and searching only parts of the network (e.g., detection head or neck),
- relying on published efficient architectures as baselines rather than performing a full search [35].
- using lightweight proxy metrics or latency predictors instead of measuring every candidate on hardware.

These strategies allow some benefits of architecture optimisation without the full computational cost of large-scale NAS [4].

Relation to model compression

NAS and compression address efficiency in complementary ways. Compression methods such as pruning and quantisation start from an existing architecture and aim to reduce its cost after training [5]. NAS, in contrast, aims to discover architectures that are efficient *by design* [4]. In practice, both approaches can be combined: an architecture designed or selected for efficiency may still benefit from pruning, quantisation, or distillation to further meet strict edge constraints. From an edge deployment perspective, this combination is often attractive because it balances model quality with practical runtime performance [34].

Summary

This chapter introduced the core building blocks used throughout the thesis: neural networks and their training, convolutional networks for vision workloads, and transformer-based models, including vision transformers, as modern alternatives for sequence-style processing in vision [18, 36, 6].

It then motivated *frugal artificial intelligence* as a practical viewpoint where accuracy must be balanced against latency, memory, energy, and data/communication constraints typical of edge deployment [34]. To meet these constraints, the chapter reviewed four main families of efficiency methods: quantisation (lower numerical precision), pruning (removing parameters/structures), knowledge distillation (teacher–student transfer), and hardware-aware NAS (architectures optimised under device-specific cost metrics). Finally, it highlighted that real deployment performance depends on the full hardware–software stack (e.g., operator support, memory bandwidth, runtime kernels), so evaluation should reflect realistic conditions rather than relying only on proxy metrics.

3. Experimental methodology

This chapter defines the experimental plan used to compare model compression strategies for object detection. The central idea is to evaluate a small but representative set of COCO-trained detectors under a shared protocol, then measure how accuracy and runtime change after applying progressively stronger efficiency methods. The study uses COCO because it is a standard object detection benchmark with established evaluation metrics and public checkpoints [21].

3.1. Research objective and validation logic

The operational research objective is to determine how different object detector families respond to practical compression methods when they are evaluated under the same COCO-oriented deployment protocol. More precisely, the thesis asks whether transformer-based detectors, hierarchical vision transformers, and YOLO-style convolutional detectors show different accuracy–latency–energy trade-offs after FP16 inference, INT8 post-training quantisation, token reduction, and light structured pruning.

The validation logic has four stages. First, a published COCO checkpoint is evaluated without compression to establish an accuracy and latency baseline. Second, a low-risk precision change (FP16) is measured to check whether the detector is numerically stable under reduced precision. Third, an INT8 deployment path is tested with calibration data and a real inference backend. Fourth, architecture-specific compression is added: token reduction for transformer-style detectors and structured pruning for YOLO-style detectors when the software stack is stable enough.

A compressed model is considered useful only if its accuracy loss is justified by a measurable deployment benefit. Therefore, the methodology does not optimise mAP alone. It compares mAP@0.5:0.95, AP@0.5, median latency, throughput, memory use, and approximate energy per image. The resulting evaluation is intended to identify Pareto-favourable configurations rather than a single universally best detector.

3.2. Hardware and practical scope

The local experimental device is an NVIDIA GeForce RTX 3070 Ti with 8 GB of GPU memory. This hardware is powerful enough for inference benchmarking, export experiments, calibration, and short fine-tuning runs, but it is not appropriate for training large COCO detectors from scratch. Therefore, the experimental scope is deliberately focused on:

- COCO validation-set evaluation of published checkpoints,
- FP32 and FP16 inference on the local GPU,
- INT8 post-training quantisation using a representative calibration subset,
- token reduction in transformer-style models,
- light structured pruning with limited fine-tuning when memory permits.

Batch size 1 is the primary latency setting because it corresponds to real-time camera-like inference. Batch size 8 is treated as a secondary throughput setting and may be skipped for the largest variants if memory usage becomes unstable.

3.3. Current implementation stage

The full methodology is broader than the set of measurements already completed. At the current stage, the implemented pipeline covers:

- RF-DETR Medium FP32 and FP16 evaluation on the full COCO 2017 validation set,
- YOLOv8 and YOLO26 FP32/FP16 inference on the COCO validation split,
- YOLOv8 and YOLO26 INT8 post-training quantisation through TensorRT engines,
- telemetry collection for latency, throughput, power, energy per image, GPU memory, utilisation, and temperature.

The remaining planned work includes Swin-T Mask R-CNN evaluation, RF-DETR INT8 export, transformer token-reduction experiments, and pruning experiments. These components require a more stable Linux-based environment because the OpenMMLab and TensorRT stacks are sensitive to Python, CUDA, PyTorch, and TensorRT versions. For this reason, the results chapter separates completed measurements from planned extensions instead of presenting the whole matrix as finished.

3.4. Dataset preparation and calibration protocol

The evaluation dataset is COCO 2017 validation. The full validation split contains 5000 images, which is used when the model/runtime combination is stable enough for unattended evaluation. For faster iteration, a 500-image validation subset is also used. This subset is not a replacement for the final COCO evaluation, but it is useful for checking whether an export path works, whether labels are mapped correctly, and whether a compression method causes an obvious accuracy collapse.

For YOLO-family models, COCO annotations are converted to the label format expected by the Ultralytics validation pipeline. The conversion preserves COCO category order and writes one text file per image. In addition to the validation subset, a representative calibration subset is prepared for INT8 post-training quantisation. The calibration images are passed through the network during TensorRT export so that activation ranges can be estimated before the final INT8 engine is built. The calibration subset is deliberately drawn from the same data distribution as the validation images, because PTQ quality depends strongly on whether the calibration inputs represent the real inference workload.

3.5. Benchmarking workflow

Each experiment row is represented by a tuple containing the model identifier, precision or compression variant, runtime backend, batch size, dataset split, and image count. The benchmark runner records both the numeric result and the raw metric file for later inspection. A completed row therefore contains at least:

- model family and checkpoint identifier,
- precision variant (FP32, FP16, INT8 PTQ),
- runtime backend (PyTorch, TensorRT, or pending backend),
- number of evaluated images,
- COCO mAP metrics,
- latency, throughput, GPU memory, power, and energy fields when available,
- path to the model artifact or exported engine.

This row-based design is intentionally simple. It allows partial overnight runs to remain useful even if a later model fails to export or a dependency stack breaks. Failed and skipped rows are kept in the summary file with an explanatory message, which is important for reproducibility because deployment failures are themselves part of the practical evaluation.

3.6. Solution architecture and implementation environment

The experimental implementation is organised as a benchmarking pipeline rather than as a single monolithic script. The pipeline contains five main components:

- a data layer that stores COCO images, annotations, converted YOLO labels, and calibration subsets,
- model adapters for RF-DETR, YOLO/YOLO26, and the planned MMDetection Swin detector,
- an export layer for precision conversion and deployment artifacts such as TensorRT engines and ONNX models,
- a benchmark runner that executes each experiment row and records accuracy, latency, and telemetry,
- an aggregation layer that writes summary CSV files and presentation/thesis plots.

Most implementation code is written in Python because the relevant model and deployment libraries expose Python APIs. The current pipeline uses PyTorch for baseline inference, the Ultralytics package for YOLO-family checkpoints and validation, the RF-DETR package for the transformer-detector baseline, TensorRT for deployment-oriented INT8 engines, pycocotools for COCO metrics and annotation handling, and NVIDIA tooling such as NVML/`nvidia-smi` for GPU telemetry. PowerShell scripts are used only for local orchestration on Windows; the final full experiment environment is planned for Linux because OpenMMLab, MMCV, MMDetection, MMDeploy, CUDA, and TensorRT are substantially easier to align there.

Existing components are reused wherever possible: published checkpoints, official validation procedures, and standard COCO metrics are preferred over custom reimplementations. The new contribution of the implementation is the unified comparison workflow around those components: consistent experiment rows, calibration handling, telemetry capture, failure logging, and result aggregation across detector families.

3.7. Runtime and export backends

The FP32 and FP16 rows are evaluated with PyTorch unless stated otherwise. FP16 is enabled as a low-risk precision reduction because it does not require calibration and is widely supported on NVIDIA GPUs. However, FP16 speedups are not guaranteed: some models are limited by memory movement, post-processing, Python overhead, or operators that do not benefit much from half precision.

INT8 rows are treated as deployment measurements rather than only numerical-format conversions. For the YOLO-family results, INT8 PTQ uses TensorRT engine export. This matters because TensorRT performs graph optimisation, operator fusion, calibration, and engine generation, so the measured latency reflects the deployment runtime rather than raw PyTorch execution. The comparison between FP32/FP16 PyTorch and INT8 TensorRT is therefore a practical deployment comparison, not a pure isolated precision comparison.

For RF-DETR and Swin, the intended final path is similar in spirit but more involved. RF-DETR requires a reliable ONNX/TensorRT export path for the transformer detector and its post-processing. Swin-T Mask R-CNN is planned through the OpenMMLab/MMDetection ecosystem, with deployment through MMDeploy or ONNX/TensorRT where feasible. These paths are postponed to the final environment because earlier Windows and Colab attempts showed sensitivity to CUDA, PyTorch, MMCV, and TensorRT versions.

3.8. Telemetry collection

Accuracy alone is insufficient for this thesis because a detector with strong mAP can still be impractical if it is slow or energy-intensive. Therefore, the benchmark records deployment-oriented quantities alongside COCO metrics. Median latency is the main timing metric for batch-size-1 inference. Throughput is computed from latency when appropriate. GPU power, utilisation, temperature, and memory are sampled during the run using NVIDIA tooling where available. Energy per image is estimated from the observed power and throughput window.

These telemetry values should be interpreted as approximate but useful. They depend on driver version, GPU clocks, cooling, background processes, and runtime implementation. Nevertheless, they reveal effects that FLOPs alone cannot capture, such as the difference between a PyTorch model and an optimised TensorRT engine.

3.9. Candidate detector families

The study compares three detector families that cover different architectural assumptions:

Family	Candidate model	Reason for inclusion
Real-time DETR	RF-DETR Nano/Medium	Detection transformer designed for strong COCO accuracy–latency trade-offs and released with ready COCO checkpoints [31].
Hierarchical ViT	Swin-T Mask R-CNN	Windowed hierarchical transformer detector with public MMDetection configs and COCO checkpoints [22].
CNN/YOLO baseline	YOLOv8s/m and YOLO26n/s/m	Dense convolutional one-stage detector line with widely used COCO checkpoints. YOLOv8 provides a historical Ultralytics baseline, while YOLO26 adds a newer NMS-free deployment-oriented comparison point [15, 16].

Table 3.1: Detector families selected for the experimental comparison.

RF-DETR Medium is the preferred local RF-DETR baseline because a checkpoint is already available in the project workspace. RF-DETR Nano should be used as a fallback if memory or export constraints make the medium variant inconvenient. For the Swin family, Swin-T Mask R-CNN is preferred over larger Swin variants because it gives the hierarchical transformer comparison without exceeding the practical limits of the local GPU. For the CNN baseline, YOLOv8m gives a strong historical YOLO baseline, while YOLO26m is the main newer YOLO comparison. YOLOv8s and YOLO26s are useful for quick iteration, and YOLO26l/x are optional upper Pareto points if the 8 GB GPU memory budget allows stable batch-size-1 evaluation.

3.10. Compression methods

The compression study varies three main dimensions.

3.10.1. Token reduction

Token reduction applies only to transformer-style detectors. The baseline implementation will use a training-free token merging strategy inspired by Token Merging (ToMe), MaMe, and Mutual

Pair Merging (MPM) [3, 13, 27]. In RF-DETR, token merging will be inserted in encoder layers before the decoder consumes image features. In Swin, token reduction will be explored either within local windows or after window attention, depending on which implementation causes the smallest disruption to the MMDetection model.

The initial token-reduction setting should be conservative: one mid-network merge point and a small merge ratio. More aggressive settings are added only after the baseline setting has been validated on a COCO subset.

3.10.2. Quantisation

The quantisation comparison includes FP32, FP16, and INT8 post-training quantisation. FP16 measures the benefit of using tensor-core-friendly reduced precision. INT8 PTQ requires calibration on a representative COCO subset before evaluating on the validation set. Quantisation-aware training is reserved as an optional extension for one model family if PTQ produces a large accuracy loss.

In the implemented YOLO pipeline, INT8 PTQ is performed by exporting TensorRT engines calibrated on COCO validation images. This is the main deployment-oriented INT8 path because it measures the runtime that would actually be used for accelerated inference. ONNX Runtime INT8 quantisation was also prepared as a fallback for producing quantised ONNX models, but it is treated separately from TensorRT because CPU ONNX Runtime latency is not directly comparable with GPU TensorRT latency.

3.10.3. Structured pruning

Structured pruning removes complete channels, filters, attention heads, or blocks, rather than individual unstructured weights. The main reason is practical: structured changes are more likely to reduce real latency on common inference backends [20]. In this thesis, pruning is treated as a light final compression step after the more deployment-friendly methods have been tested.

3.11. Experiment matrix

The full design is intentionally larger than the minimum required result set. This makes it possible to stop early while still preserving a coherent thesis.

Variant	Priority	Purpose
FP32 baseline	Required	Establish published-checkpoint accuracy and local latency.
FP16 inference	Required	Measure low-risk acceleration on the RTX 3070 Ti.
INT8 PTQ	Required	Measure post-training quantisation accuracy loss and speedup.
Token reduction only	Required for RF-DETR/Swin	Isolate the effect of reducing token count.
Token reduction + FP16	Recommended	Test whether token reduction and mixed precision compose cleanly.
Token reduction + INT8 PTQ	Required for RF-DETR/Swin	Main transformer compression result.
Token reduction + INT8 PTQ + pruning	Optional	Highest compression setting if time remains.
Alternative method order	Optional	Test the Progressive Intensity hypothesis for object detection [17].

Table 3.2: Compression variants planned for each detector family.

For YOLO, the token-reduction rows are replaced by structured pruning rows because token merging is not a natural operation for a convolutional detector. The key comparison is therefore not identical operations in every model, but matched compression intent: reduce representation size, reduce numerical precision, and measure the resulting accuracy–latency trade-off.

3.12. Evaluation metrics

Each model and compression variant will be evaluated using the following metrics:

- COCO mAP@[0.5:0.95] as the primary accuracy metric,
- AP@0.5 as a more permissive detection-quality metric,
- latency in milliseconds for batch size 1 and, where feasible, batch size 8,
- parameters and model file size,
- FLOPs or MACs, reported as a proxy metric rather than a replacement for measured latency,

- peak GPU memory during inference,
- optional power draw or energy per image from repeated benchmark windows.

Latency measurements will use warm-up iterations followed by repeated timed iterations with CUDA synchronisation. Results will be reported as median latency and, where possible, an interquartile range. This avoids over-interpreting single measurements affected by background processes or GPU clock changes.

3.13. Robustness extension

If the main matrix is completed early, a robustness extension can evaluate a subset of compressed detectors under controlled input degradations: Gaussian noise, blur, and JPEG compression. This extension is useful because quantisation and token reduction may change not only average accuracy, but also how quickly a detector degrades when image quality becomes worse.

3.14. Threats to validity

Several limitations must be documented. First, RF-DETR, Swin Mask R-CNN, and YOLO differ not only in backbone design, but also in detection heads, training recipes, input resolutions, and post-processing. Second, latency depends strongly on the inference runtime, exported graph, driver version, and operator support. Third, an RTX 3070 Ti is a useful local deployment proxy, but it is not the same as an embedded edge accelerator. For these reasons, conclusions should emphasise measured trade-offs under the stated protocol rather than universal claims about every deployment setting.

4. Preliminary results

This chapter reports the measurements completed so far. The results are preliminary: they validate the benchmarking and export workflow, but they do not yet cover the full final matrix described in Chapter 3. The reported RF-DETR, YOLOv8, and YOLO26 accuracy values are presented as COCO 2017 validation measurements on the full 5000-image validation split. A separate 500-image subset is used for INT8 calibration and rapid smoke tests, but it is not the basis of the reported mAP values in the main result tables.

4.1. Completed experiment scope

The completed measurements cover two parts of the planned comparison. First, RF-DETR Medium was evaluated in FP32 and FP16 precision using the PyTorch runtime on the full COCO validation set. Second, YOLOv8s, YOLOv8m, YOLO26s, and YOLO26m were evaluated in FP32, FP16, and INT8 TensorRT precision with the same COCO validation target. The Swin-T Mask R-CNN part of the matrix and transformer-specific token-reduction experiments remain planned work.

The reported latency is median per-image latency at batch size 1. For the YOLO INT8 rows, the runtime is TensorRT and the engines were produced through post-training quantisation with a COCO calibration subset. The raw result artifacts are retained under the project `runs/` directory together with exported model artifacts and plot files used in the presentation.

4.2. Data used in the preliminary experiments

The preliminary experiments use the COCO 2017 validation split. The full validation set contains 5000 images and an annotation file in JSON format. Each image entry contains fields such as image identifier, file name, width, and height. Each object annotation contains an image identifier, category identifier, bounding box, segmentation metadata, object area, and an `iscrowd` flag. The most important numeric fields for this thesis are the bounding boxes and category

identifiers, because they are used to compute COCO mAP and AP@0.5.

For YOLO-family validation, the COCO annotations are transformed into the Ultralytics label format. In this format, each image has a corresponding text file containing class identifiers and normalised bounding boxes. The transformation changes the representation of the labels but not the evaluation target: the model is still evaluated against COCO object categories. The benchmark also creates a smaller calibration subset for INT8 PTQ. Calibration images are used only to estimate activation ranges during export; they are not treated as training data and are not used to report final accuracy.

No detector is trained from scratch in the preliminary stage. The models use published COCO-trained checkpoints, so the COCO training split is part of the original checkpoint training process rather than part of the local experiment. Locally, the validation split is used in two ways: the full 5000-image set for the reported accuracy measurements, and a 500-image subset for calibration or rapid export checks. This distinction matters because the calibration subset is an implementation input for INT8 PTQ, while the full validation split is the benchmark target for reported mAP.

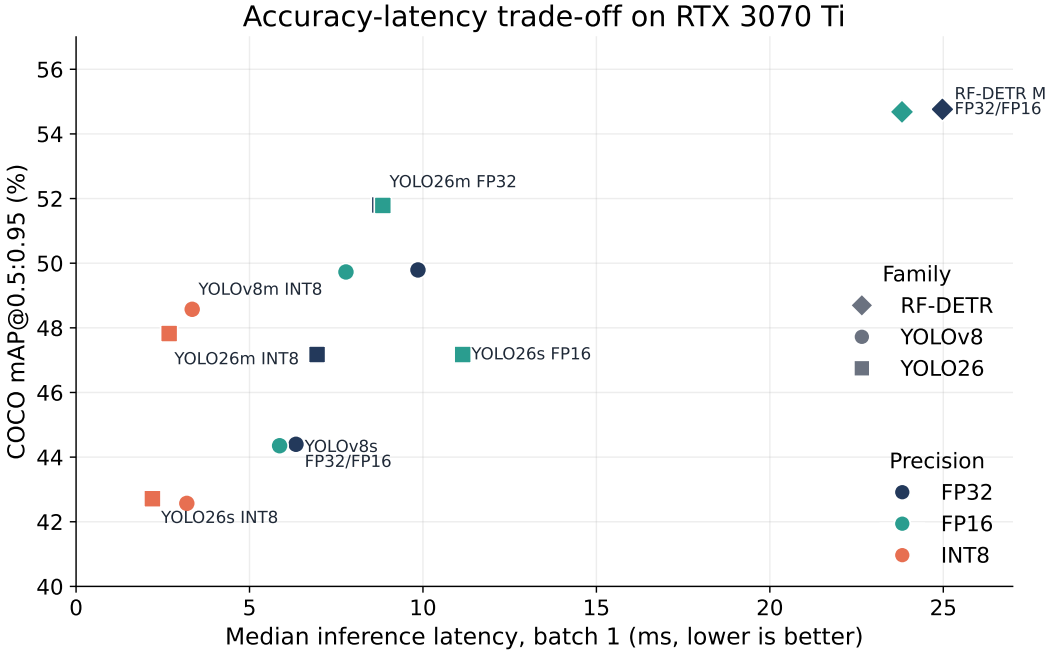


Figure 4.1: Accuracy–latency overview of the completed preliminary measurements. Accuracy values are reported against the COCO 2017 validation target; the separate 500-image subset is used for INT8 calibration and smoke tests, not as the main accuracy benchmark.

Figure 4.1 summarises the current state of the experiment matrix. The RF-DETR points occupy the high-accuracy but higher-latency region. YOLO and YOLO26 points occupy the lower-latency region, especially after TensorRT INT8 export. The figure also shows why a single

metric is insufficient: RF-DETR is currently the strongest measured detector in accuracy, while several YOLO-family INT8 engines are much faster at the cost of lower mAP.

4.3. RF-DETR baseline

Model	Precision	Images	mAP@0.5:0.95 [%]	AP@0.5 [%]	Latency [ms]
RF-DETR Medium	FP32	5000	54.76	73.63	24.97
RF-DETR Medium	FP16	5000	54.68	73.66	23.81

Table 4.1: Preliminary RF-DETR Medium results on the full COCO 2017 validation set.

The RF-DETR Medium baseline achieved the highest accuracy among the completed measurements, reaching 54.76% mAP@0.5:0.95 in FP32. Switching to FP16 caused almost no accuracy loss: the measured mAP changed by less than 0.1 percentage point. The latency improvement was modest, from 24.97 ms to 23.81 ms per image. This suggests that, in the current PyTorch path, RF-DETR is not yet strongly accelerated by FP16 alone. A more deployment-oriented conclusion will require TensorRT or another optimised inference backend.

The small difference between FP32 and FP16 is still useful. It suggests that the RF-DETR checkpoint itself is numerically stable under half precision. Therefore, future work can focus on export and runtime optimisation rather than first solving a basic precision-stability problem. The next RF-DETR experiments should prioritise ONNX/TensorRT export, INT8 calibration, and then token reduction in the encoder.

4.4. YOLO and YOLO26 precision comparison

Model	Precision/runtime	Images	mAP@0.5:0.95 [%]	AP@0.5 [%]	Latency [ms]	Speedup vs FP32
YOLOv8s	FP32/PyTorch	5000	44.40	61.05	6.34	1.00
YOLOv8s	FP16/PyTorch	5000	44.35	61.06	5.87	1.08
YOLOv8s	INT8/TensorRT	5000	42.57	59.05	3.19	1.99
YOLOv8m	FP32/PyTorch	5000	49.79	66.55	9.86	1.00
YOLOv8m	FP16/PyTorch	5000	49.73	66.53	7.78	1.27
YOLOv8m	INT8/TensorRT	5000	48.57	65.25	3.35	2.94
YOLO26s	FP32/PyTorch	5000	47.17	63.85	6.95	1.00
YOLO26s	FP16/PyTorch	5000	47.17	63.85	11.15	0.62
YOLO26s	INT8/TensorRT	5000	42.71	59.39	2.20	3.16
YOLO26m	FP32/PyTorch	5000	51.80	69.06	8.76	1.00
YOLO26m	FP16/PyTorch	5000	51.79	69.06	8.84	0.99
YOLO26m	INT8/TensorRT	5000	47.82	65.98	2.68	3.26

Table 4.2: Preliminary YOLO and YOLO26 results on the COCO 2017 validation target.

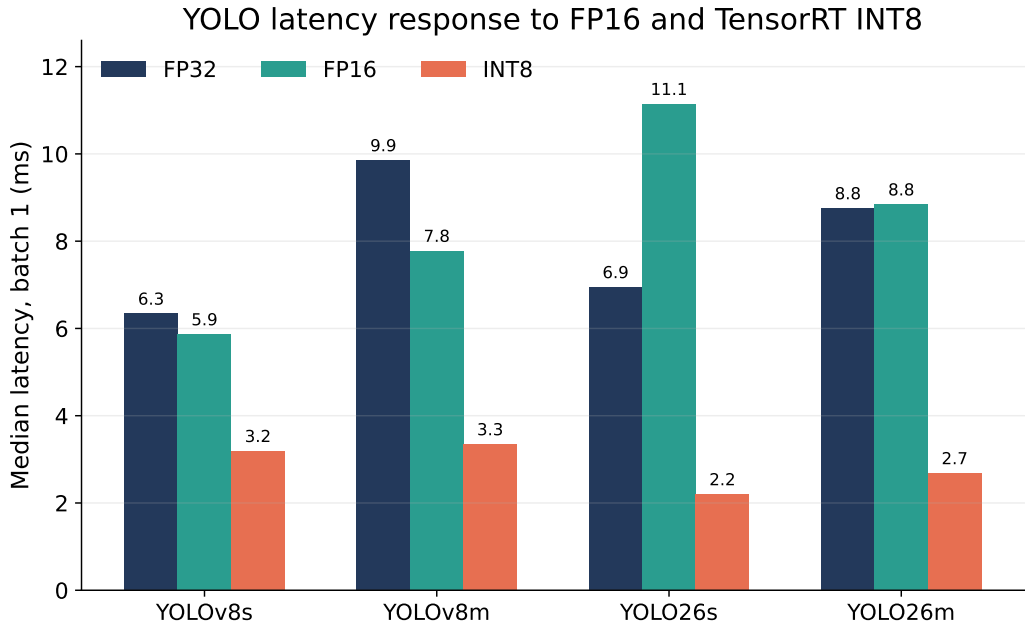


Figure 4.2: Median batch-size-1 latency for YOLO-family models under FP32, FP16, and TensorRT INT8.

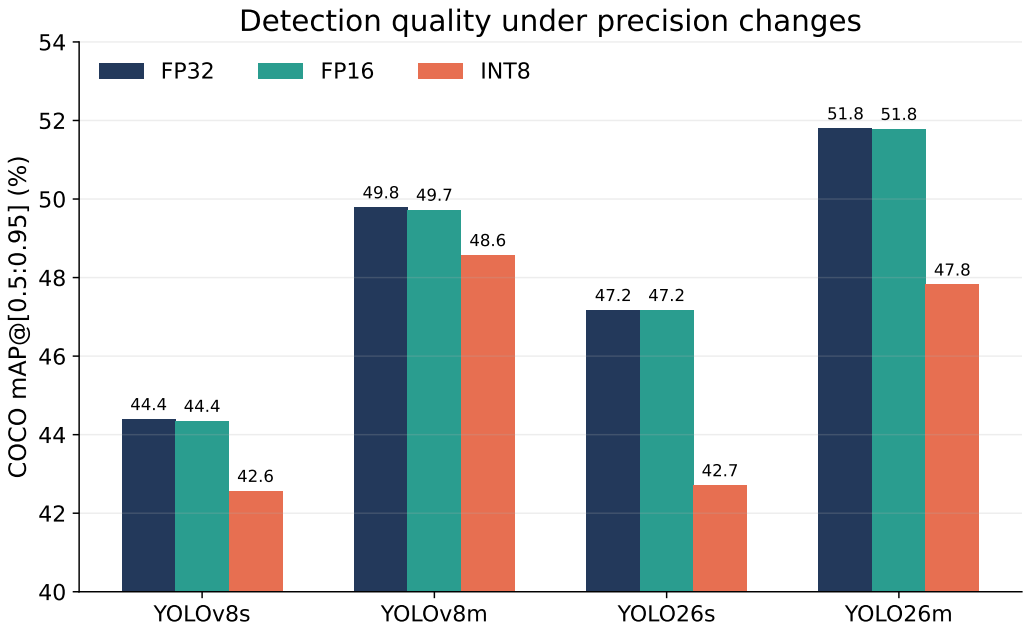


Figure 4.3: Detection quality for YOLO-family models under precision changes on the COCO 2017 validation target.

The YOLO-family results show the expected trade-off between accuracy and speed for INT8 TensorRT inference. YOLOv8m lost 1.22 percentage points of mAP compared with FP32 while improving median latency from 9.86 ms to 3.35 ms, which corresponds to a 2.94× speedup. YOLOv8s showed a similar pattern: INT8 reduced mAP by 1.83 percentage points and almost doubled speed relative to FP32.

YOLO26 produced the strongest FP32 accuracy among the YOLO-family rows. YOLO26m reached 51.80% mAP, compared with 49.79% for YOLOv8m. Its INT8 TensorRT version was also fast, reaching 2.68 ms per image, but the accuracy drop was larger than for YOLOv8m. YOLO26s was the fastest INT8 model in the current measurements at 2.20 ms per image, but it also showed a larger mAP reduction after INT8 quantisation. These results suggest that the newer YOLO26 models are promising for latency, but their PTQ behaviour should be investigated more carefully with repeated full-validation runs and, if time permits, quantisation-aware training.

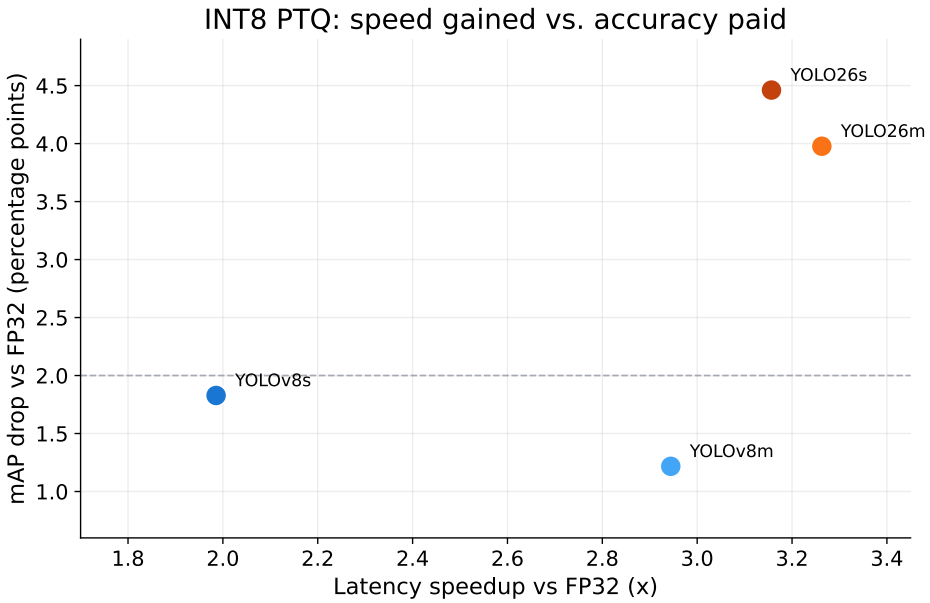


Figure 4.4: INT8 TensorRT speedup relative to FP32 versus mAP loss for YOLO-family models. Lower and further right is preferable: more speedup with less accuracy loss.

Figure 4.2 shows that TensorRT INT8 consistently provides the largest latency reduction among the completed YOLO experiments. The FP16 results are less uniform. YOLOv8s and YOLOv8m become faster under FP16, but YOLO26s becomes slower and YOLO26m remains almost unchanged. This indicates that simply enabling half precision is not enough to guarantee speedup; the model architecture, post-processing path, and runtime implementation still determine the measured latency. The YOLO26s latency regression under FP16 is especially important: latency increases from 6.95 ms in FP32 to 11.15 ms in FP16. This suggests that an architectural layer, activation format, or post-processing path may lack an optimised half-precision kernel, or may trigger an inefficient fallback in the Windows PyTorch environment. The observation supports the use of deployment frameworks such as TensorRT when speedups must be guaranteed rather than assumed.

Figure 4.3 shows a different pattern for accuracy. FP16 preserves mAP almost exactly for all tested YOLO-family models, whereas INT8 PTQ introduces a visible accuracy drop. The magnitude of this drop differs by model: the YOLOv8 models lose less mAP than the YOLO26 models in the current result set. Figure 4.4 makes this trade-off explicit. YOLOv8m is currently the most balanced INT8 point among the tested models, because it gains nearly a 3× speedup with only about 1.2 percentage points of mAP loss. YOLO26m and YOLO26s are faster in INT8, but they pay a larger accuracy cost.

4.5. Energy and deployment observations

Model	Precision/runtime	Latency [ms]	Power [W]	Energy [J/image]
RF-DETR Medium	FP32/PyTorch	24.97	136.93	4.29
RF-DETR Medium	FP16/PyTorch	23.81	91.21	2.65
YOLOv8m	FP32/PyTorch	9.86	138.90	21.82
YOLOv8m	FP16/PyTorch	7.78	124.08	16.24
YOLOv8m	INT8/TensorRT	3.35	89.24	8.26
YOLO26m	FP32/PyTorch	8.76	142.30	20.05
YOLO26m	FP16/PyTorch	8.84	116.83	16.47
YOLO26m	INT8/TensorRT	2.68	89.09	7.72

Table 4.3: Selected telemetry values from the preliminary runs. Energy estimates are approximate and depend on the sampled benchmark window.

The telemetry collected during the same runs also indicates that runtime choice can strongly affect energy per image. For example, YOLOv8m decreased from 21.82 J/image in FP32 PyTorch to 8.26 J/image in INT8 TensorRT, while YOLO26m decreased from 20.05 J/image to 7.72 J/image. RF-DETR also benefited from FP16 in energy terms, decreasing from 4.29 J/image to 2.65 J/image, even though its latency changed only slightly. This supports the thesis assumption that latency, energy, and accuracy should be measured together rather than inferred from FLOPs alone.

One interesting observation is that lower latency and lower power can occur together when the runtime is better optimised. The TensorRT INT8 engines both reduce inference time and reduce measured power relative to PyTorch FP32 for the medium YOLO variants. This compounds the energy benefit. In contrast, FP16 sometimes reduces power without reducing latency much, as seen for YOLO26m. Such cases are still relevant for deployment because energy and thermal behaviour can matter even when latency is unchanged.

4.6. Implications for the next experimental stage

The preliminary results motivate three concrete next steps. First, YOLO-family experiments should be repeated in the final Linux environment to confirm that the observed INT8 accuracy drops and FP16 latency behaviour are stable across runs and software stacks. Second, RF-DETR should be exported to an optimised backend before conclusions are drawn about transformer

detector latency. The PyTorch RF-DETR numbers are useful as a baseline, but they do not yet represent the best deployment path. Third, Swin-T Mask R-CNN should be added once the OpenMMLab stack is stable, because it is the missing hierarchical transformer point needed to complete the architectural comparison.

The results also refine the compression plan. FP16 can be treated as a low-risk baseline because it preserves accuracy across the completed experiments. INT8 PTQ is more powerful but model-dependent: it provides large latency and energy gains, but the mAP loss can vary substantially. Token reduction and pruning should therefore be evaluated after stable FP32/FP16/INT8 baselines are available, otherwise it becomes difficult to attribute accuracy loss to a particular compression step.

4.7. Error sources and uncertainty

The main source of statistical uncertainty is not the validation target itself, but the small number of repeated timing runs and the evolving software environment. COCO mAP is reported against the validation target, while the 500-image subset is used for calibration and smoke testing. The full thesis evaluation should still repeat the YOLO and YOLO26 measurements in the final Linux environment and, where possible, report latency variation across repeated runs.

There is also implementation uncertainty. The FP32 and FP16 results use PyTorch, while the INT8 results use TensorRT. This is appropriate for deployment evaluation, but it means that speedups reflect both lower numerical precision and backend optimisation. TensorRT version changes can also affect export success, layer fusion, calibration behaviour, and latency. This was already visible in the development process, where some INT8 export paths worked in one Colab/runtime configuration but failed in another.

Energy measurements are approximate because they are derived from sampled GPU telemetry rather than from an external power meter. They are useful for comparing broad runtime trends, especially when the same machine and protocol are used, but they should not be interpreted as exact physical measurements. Finally, Swin and token-reduction results are not yet present, so architectural conclusions about hierarchical ViTs and token compression remain hypotheses until the final experiment matrix is completed.

4.8. Current limitations

The current results should be interpreted cautiously. Although the reported accuracy values are aligned to the COCO validation target, the FP16 and INT8 rows use different runtimes: FP16 measurements were PyTorch-based, whereas INT8 measurements used TensorRT engines. As a result, part of the observed speedup comes from the deployment runtime and graph optimisation, not only from lower numerical precision. The YOLO26s FP16 regression also shows that runtime-specific kernel support can dominate the expected benefit of reduced precision. Finally, Swin Transformer results are not yet included because the OpenMMLab stack was unstable in the available Windows/Colab environments. A Linux installation is planned for the final version of the experiments.

Bibliography

[1] Areeba Abid, Priyanshu Sinha, Aishwarya Harpale, Judy Gichoya, and Saptarshi Purkayastha. Optimizing medical image classification models for edge devices. In Kenji Matsui, Sigeru Omatu, Tan Yigitcanlar, and Sara Rodríguez González, editors, *Distributed Computing and Artificial Intelligence, Volume 1: 18th International Conference*, volume 327 of *Lecture Notes in Networks and Systems*, pages 77–87. Springer, Cham, 2022.

[2] Aphex34. Typical cnn.png, December 2015. Image. Source: Own work. Licensed under CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0/).

[3] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your ViT but faster. In *International Conference on Learning Representations (ICLR)*, 2023.

[4] Han Cai, Ligeng Zhu, and Song Han. Proxylessnas: Direct neural architecture search on target task and hardware. In *International Conference on Learning Representations (ICLR)*, 2019.

[5] Yu Cheng, Duo Wang, Pan Zhou, and Tao Zhang. A survey of model compression and acceleration for deep neural networks. *arXiv preprint arXiv:1710.09282*, 2017.

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[7] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. *arXiv preprint arXiv:2103.13630*, 2021.

[8] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, 2014.

[9] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations (ICLR)*, 2016.

[10] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[11] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[12] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

[13] Simin Huo and Ning Li. MaMe and MaRe: Matrix-based token merging and restoration for efficient visual perception and synthesis, 2026. Preprint.

[14] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew T. H. Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[15] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8, 2023. Software and model documentation.

[16] Glenn Jocher and Jing Qiu. Ultralytics YOLO26, 2026. Software and model documentation. Released January 14, 2026.

[17] Minjun Kim, Jaehyeon Choi, Hyunwoo Yang, Jongjin Kim, Jinho Song, and U. Kang. Prune-then-quantize or quantize-then-prune? understanding the impact of compression order in joint model compression, 2026.

[18] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

[19] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[20] Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. Pruning filters for efficient convnets. In *International Conference on Learning Representations (ICLR)*, 2017.

[21] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, 2014.

[22] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[23] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.

[24] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943.

[25] Andrew Miller, Fabio Di Troia, and Mark Stamp. An empirical analysis of hidden markov models with momentum. In Mark Stamp and Martin Jureček, editors, *Machine Learning, Deep Learning and AI for Cybersecurity*, pages 169–206. Springer, Cham, 2025.

[26] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.

[27] Simon Ravé, Pejman Rasti, and David Rousseau. MPM: Mutual pair merging for efficient vision transformers, 2026. Preprint.

[28] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[29] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, 2015.

[30] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.

[31] Isaac Robinson, Peter Robicheaux, Matvei Popov, Deva Ramanan, and Neehar Peri. RF-DETR: Neural architecture search for real-time detection transformers, 2025.

[32] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[33] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[34] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel Emer. Efficient processing of deep neural networks: A tutorial and survey. *arXiv preprint arXiv:1703.09039*, 2017.

[35] Mingxing Tan, Ruoming Pang, and Quoc V. Le. Efficientdet: Scalable and efficient object detection. *arXiv preprint arXiv:1911.09070*, 2020.

[36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

List of Figures